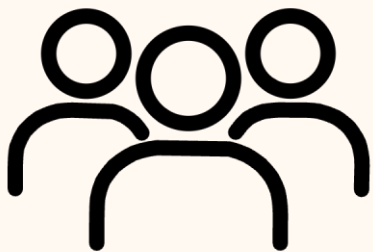




Project: Federated Learning Simulation Using Flower (FLWR)

Parallel & Distributed Computing project demonstrating a federated training pipeline with Flower, comparisons to centralized learning, and experimental findings using MNIST.



Group Members

Maryam Tariq	(23-BCS-062)
Warda Nayab	(23-BCS-091)
Hamna Qandeel	(23-BCS-038)
Faryal Asif	(23-BCS-030)
Hamna Noor	(23-BCS-037)



Submitted To: Mam Madiha Wahid

Agenda

1 Introduction

3 Problem Statement

5 System Requirements

7 Dataset Description

9 Experimental Results

11 Conclusion

2 Background of Federated Learning

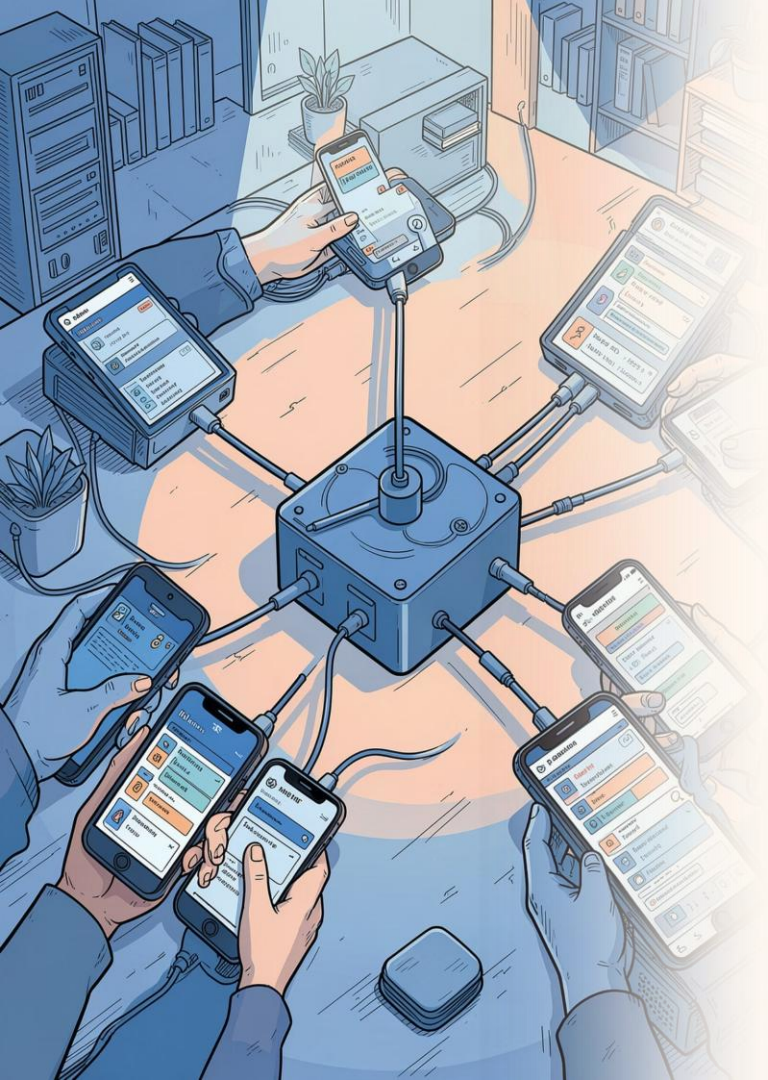
4 Objectives

6 Project Implementation

8 Neural Network Architecture

10 Comparison Analysis

12 Future Scope



Introduction to Federated Learning

- Traditional machine learning collects all data in one central server.
- This creates privacy and security concerns.
- Federated Learning allows model training without sharing raw data.
- Only model updates are shared with the central server.
- It is widely used in healthcare, finance, and mobile applications.

Background

1

Problems in Traditional Machine Learning

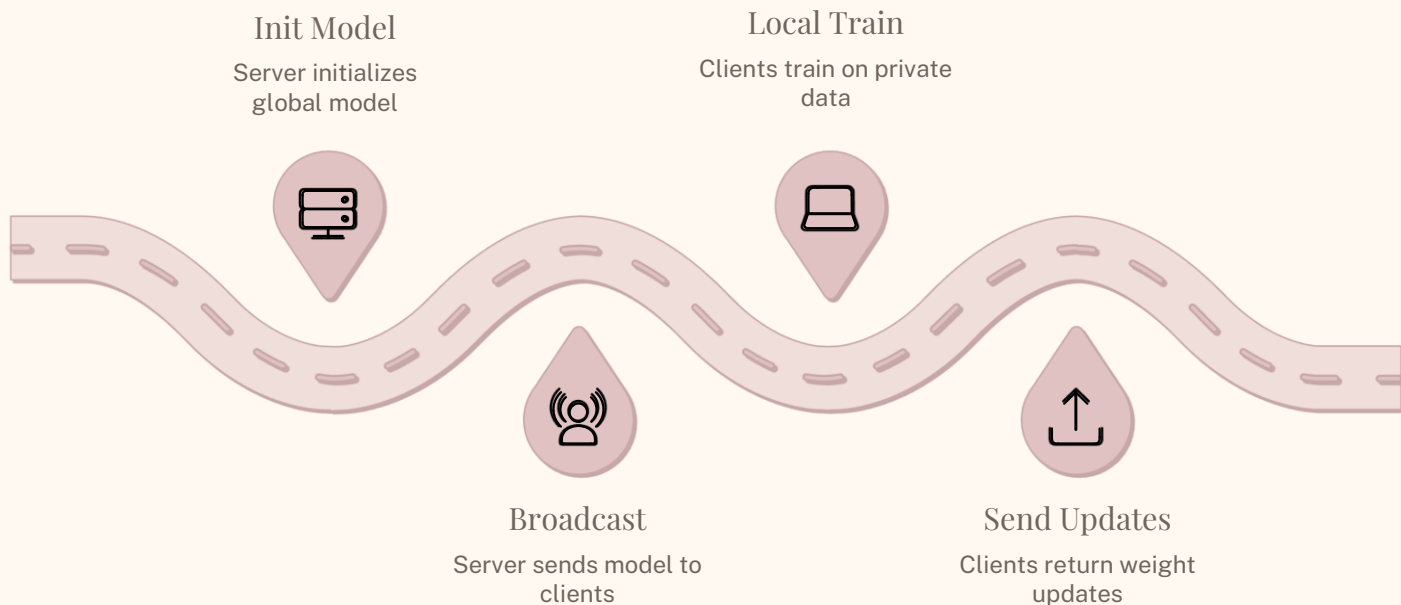
- Privacy risks
- High data transfer cost
- Regulatory issues (GDPR, HIPAA)
- Single point of failure

2

Federated Learning Solution

- Data remains on local devices
- Models train locally
- Central server aggregates updates
- Better privacy preservation

Federated Learning Workflow (High-level)



Each training round: broadcast → local updates → secure aggregation → model update. FedAvg (weighted averaging by client data size) is the most common aggregation strategy used in our simulation.

Problem Statement

This project aims to:

Implement Federated Learning using Flower framework

Simulate distributed learning with 5 clients

Train neural network on MNIST dataset

Compare Federated Learning with Centralized Training

Analyze accuracy, privacy, and training performance

Project Objectives

1

Install and configure Flower framework

2

Implement distributed client-server training

3

Apply Federated Averaging (FedAvg)

4

Train model using MNIST dataset

5

Compare federated and centralized approaches

6

Evaluate accuracy and privacy trade-offs

System Requirements

Technologies & Tools Used



Software

Python

TensorFlow

Flower (FLwr)

NumPy

Scikit-learn

Matplotlib



Version

3.10.11

2.13.0

1.6.0

1.24.3

1.3.0

3.7.2



Purpose

Programming

Deep Learning

Federated Learning

Numerical Operations

Data Processing

Visualization

Implementation Overview

1

Install Python

2

Create virtual environment

3

Install required libraries

4

Create project files

5

Run centralized training

6

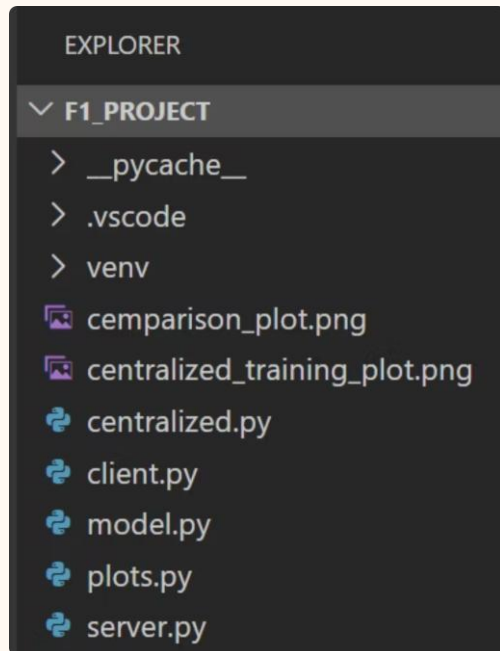
Run federated learning
simulation

7

Generate comparison graphs

Project File Structure

Project Files



Important Python Files

model.py

Neural network model

centralized.py

Traditional centralized training

client.py

Federated client implementation

server.py

Federated server using FedAvg

plots.py

Accuracy comparison graphs

Centralized Training



Working

- All MNIST data stored in one place
- Single model trained on complete dataset
- Faster training process
- No privacy preservation



Results

- Test Accuracy: 97.62%
- Training Time: 45 seconds

Federated Learning Process



Configuration

- 5 simulated clients
- 10 communication rounds
- FedAvg aggregation strategy
- Local training on each client



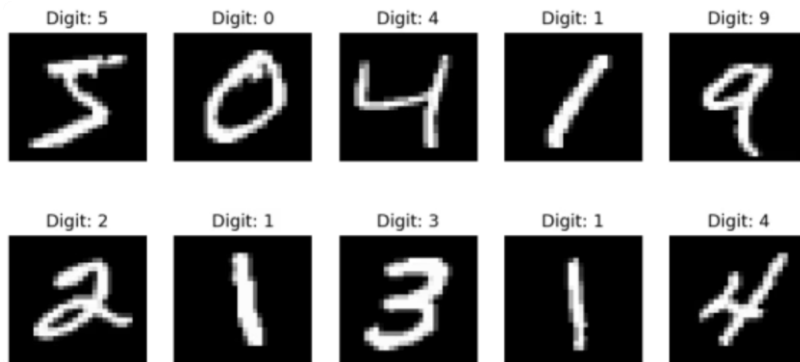
Workflow

- Server waits for clients
- Clients train locally
- Server aggregates model updates
- Global model improves after every round

MNIST Dataset

Dataset Details

Property	Value
Dataset	MNIST
Classes	Digits 0–9
Training Samples	60,000
Test Samples	10,000
Image Size	28 × 28
Format	Grayscale



Purpose

Used for handwritten digit classification.

Data Distribution

Client Data Distribution



Distribution Strategy

- Dataset divided equally among 5 clients
- Each client receives unique training data
- Simulates real-world distributed learning

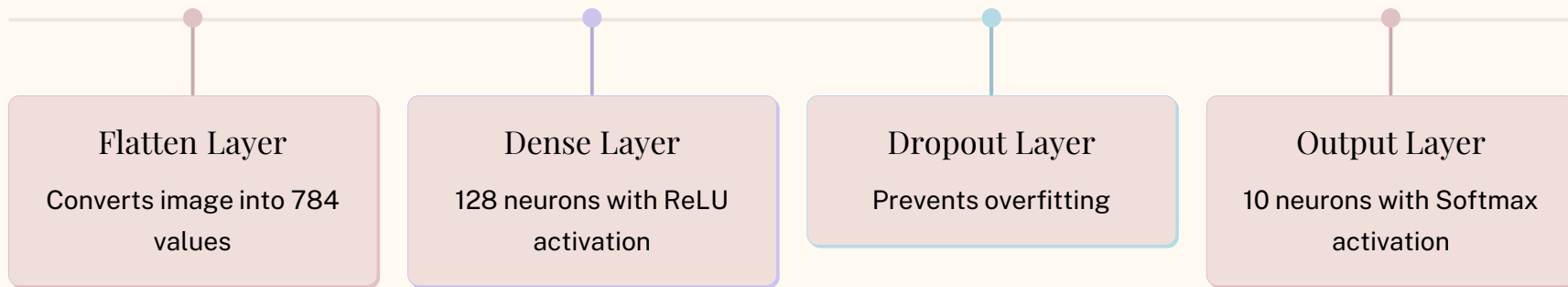


Advantages

- Data privacy maintained
- No raw data sharing
- Independent local training

Neural Network Architecture

Layers Used



Total Parameters

101,770 trainable parameters

Training Hyperparameters

Optimizer

Adam - Adapts learning rate during training for faster convergence

Loss Function

Sparse Categorical Crossentropy -
Measures difference between predicted and actual digit labels

Batch Size

32 - Processes 32 images before updating model weights

Dropout Rate

0.1 - Randomly drops 10% of neurons to prevent overfitting

Learning Rate

0.001 - Default Adam optimizer learning rate

Centralized Training

5 epochs

Federated Learning

3 local epochs per round, with 10 communication rounds

Centralized Training Results

Performance Metrics

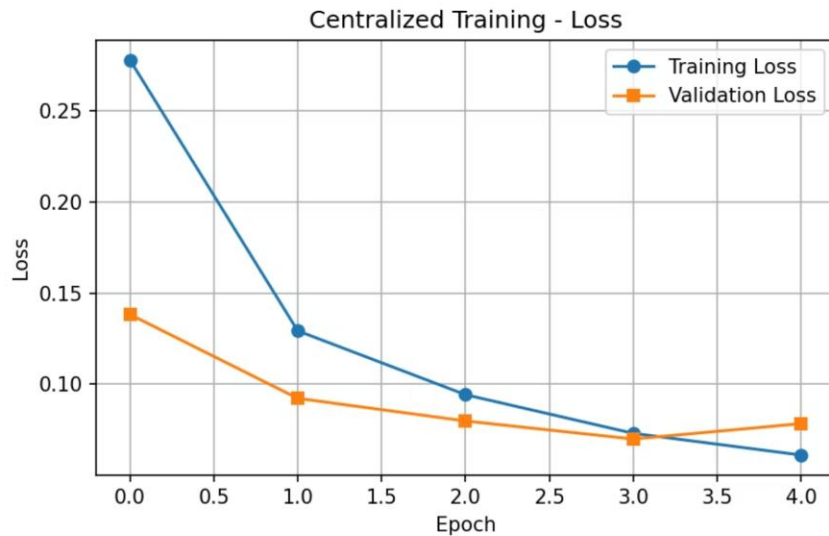
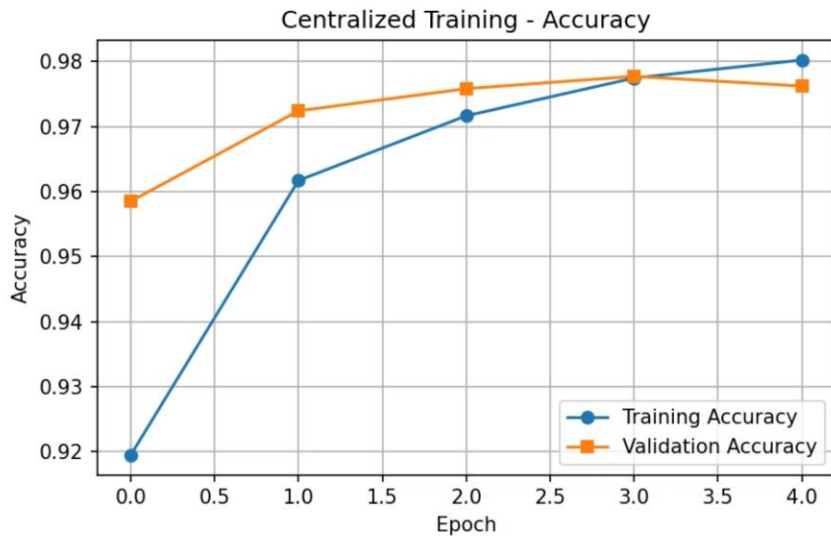
Property	Value
Training Accuracy	98.02%
Validation Accuracy	97.62%
Test Accuracy	97.62%
Test Loss	0.0784
Training Time	44.97 seconds

Epoch-wise Progress

Epoch	Training Accuracy	Validation Accuracy	Loss
1	93.18%	94.27%	0.2443
2	96.34%	95.73%	0.1257
3	97.43%	96.58%	0.0880
4	97.94%	97.09%	0.0683
5	98.02%	97.62%	0.0784

Centralized Training Results

Accuracy Curves and Loss Curves



Federated Learning Results (FedAvg)

Configuration

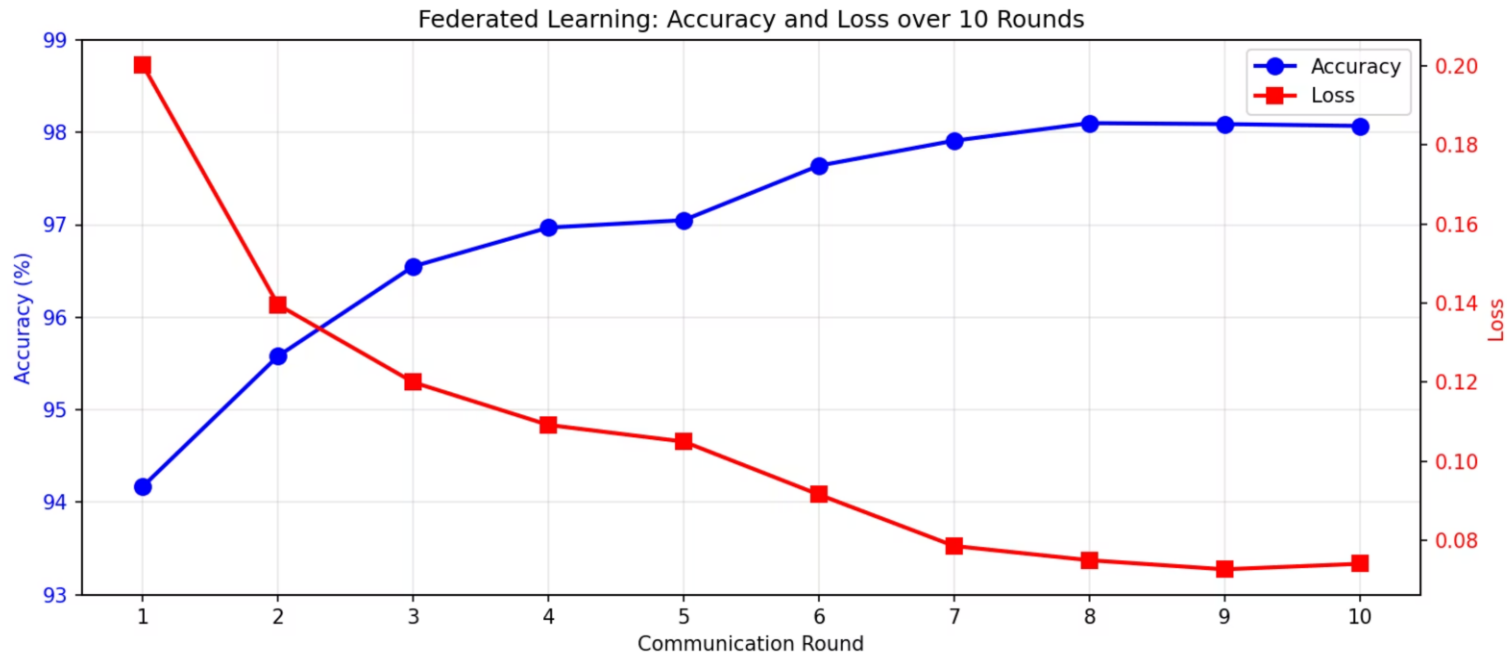
- **Strategy:** Federated Averaging (FedAvg)
- **Number of Clients:** 5
- **Communication Rounds:** 10
- **Clients Sampled Per Round:** 2 to 5 (variable)
- **Local Epochs Per Client:** 3
- **Total Training Time:** 210 seconds

Round-wise Accuracy

Round	Accuracy	Improvement	Loss
Round 1	94.17%	—	0.0583
Round 2	95.58%	+1.41%	0.0442
Round 3	96.21%	+0.63%	0.0379
Round 4	96.74%	+0.53%	0.0326
Round 5	97.08%	+0.34%	0.0292
Round 6	97.41%	+0.33%	0.0259
Round 7	97.66%	+0.25%	0.0234
Round 8	97.88%	+0.22%	0.0212
Round 9	98.09%	+0.21%	0.0191
Round 10	98.07%	-0.02%	0.0193

Federated Learning Results (FedAvg)

Accuracy and Loss Over 10 Rounds



Centralized vs Federated Learning

Metric	Centralized Learning	Federated Learning (FedAvg)
Best Accuracy	97.62%	98.10%
Training Time	45 seconds	210 seconds
Data Privacy	No	Yes
Communication Rounds	Not Required	10 Rounds
Number of Clients	1	5
Local Training	No	3 Epochs per Round

Key Insights

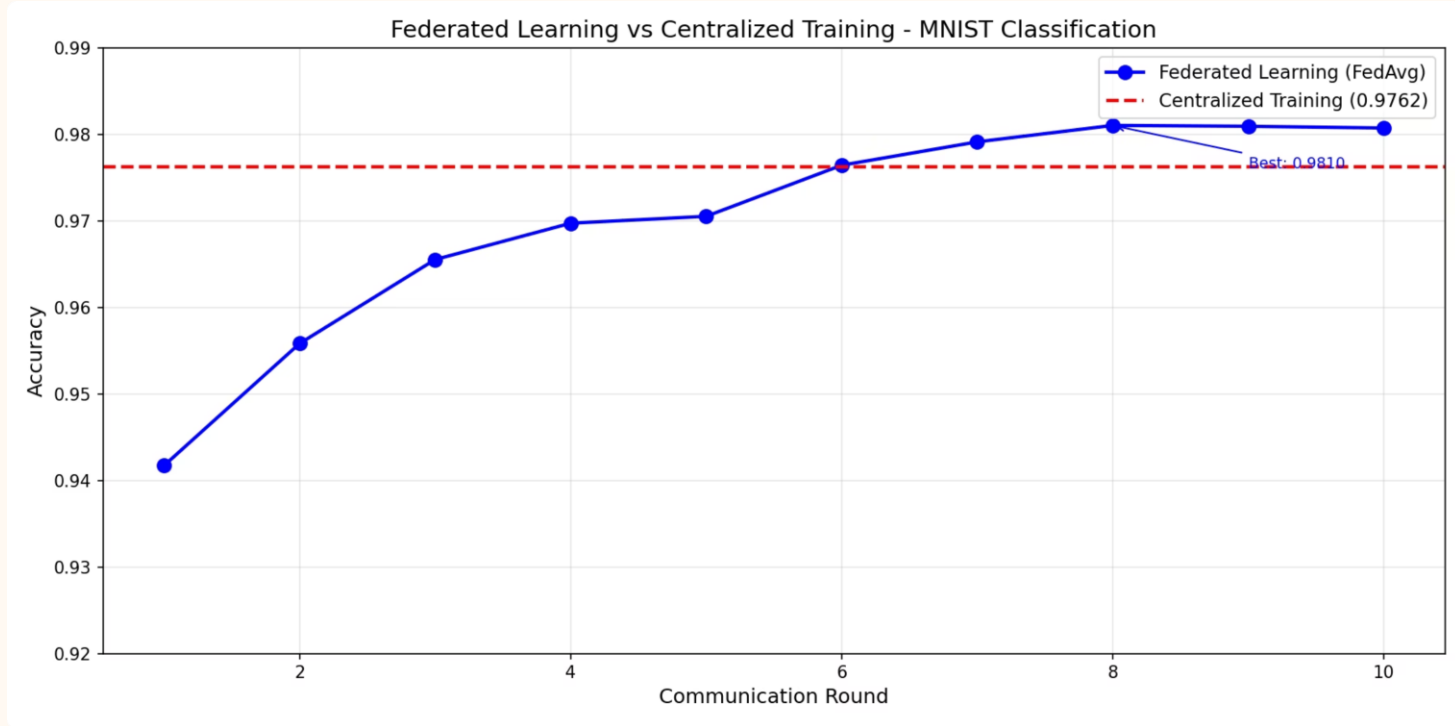
- Federated Learning achieved 0.48% higher accuracy.
- Centralized Training was 4.67× faster.
- Federated Learning provided better privacy protection.
- FedAvg performed better overall in distributed training.



Final Observation

Federated Learning achieved better overall performance while maintaining data privacy.

Comparison Analysis: Centralized vs Federated Learning



Advantages of Federated Learning



Better privacy protection

Ensures sensitive data remains on local devices, enhancing user privacy.



No raw data sharing

Only model updates, not raw data, are exchanged with the central server.



Improved global model robustness

Training on diverse client data leads to a more generalizable and resilient global model.



Suitable for distributed systems

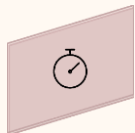
Optimized for scenarios where data is inherently spread across many devices or locations.



Useful for healthcare and finance applications

Ideal for sectors with strict data regulations and a need to leverage siloed data.

Limitations of Federated Learning



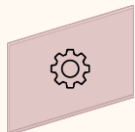
Higher training time



Communication overhead



Requires synchronization between clients



More complex implementation



Dependent on client availability

Conclusion

This project successfully explored Federated Learning through a practical implementation and comparative analysis.

Implemented with Flower

Successfully implemented Federated Learning using the Flower framework.

+0.48% Improvement

Federated Learning outperformed centralized training by 0.48%.

5 Distributed Clients

Simulated 5 distributed clients to mirror a real federated setting.

Privacy Preserved

Privacy preservation is the major advantage of this approach.

98.10% Accuracy

FedAvg strategy achieved strong performance with 98.10% accuracy.

Best for Sensitive Data

Federated Learning is ideal for sensitive data applications.

Future Scope



Implement FedProx Strategy

Explore FedProx as an advanced aggregation strategy to improve model stability and convergence.



Use Larger Datasets

Train and evaluate with larger datasets to better assess performance and generalization.



Deploy on Real Distributed Devices

Move beyond simulation and test the framework on actual distributed devices in realistic conditions.



Increase Number of Clients

Scale the simulation to include more clients for a closer representation of real federated environments.



Apply Secure Aggregation

Use secure aggregation techniques to strengthen privacy during model updates and parameter sharing.



Improve Communication Efficiency

Reduce communication overhead to make federated training faster and more practical at scale.

Thank You!

Questions & Answers

